11: Direct Memory Access (DMA)

Engr 315: Hardware / Software Codesign

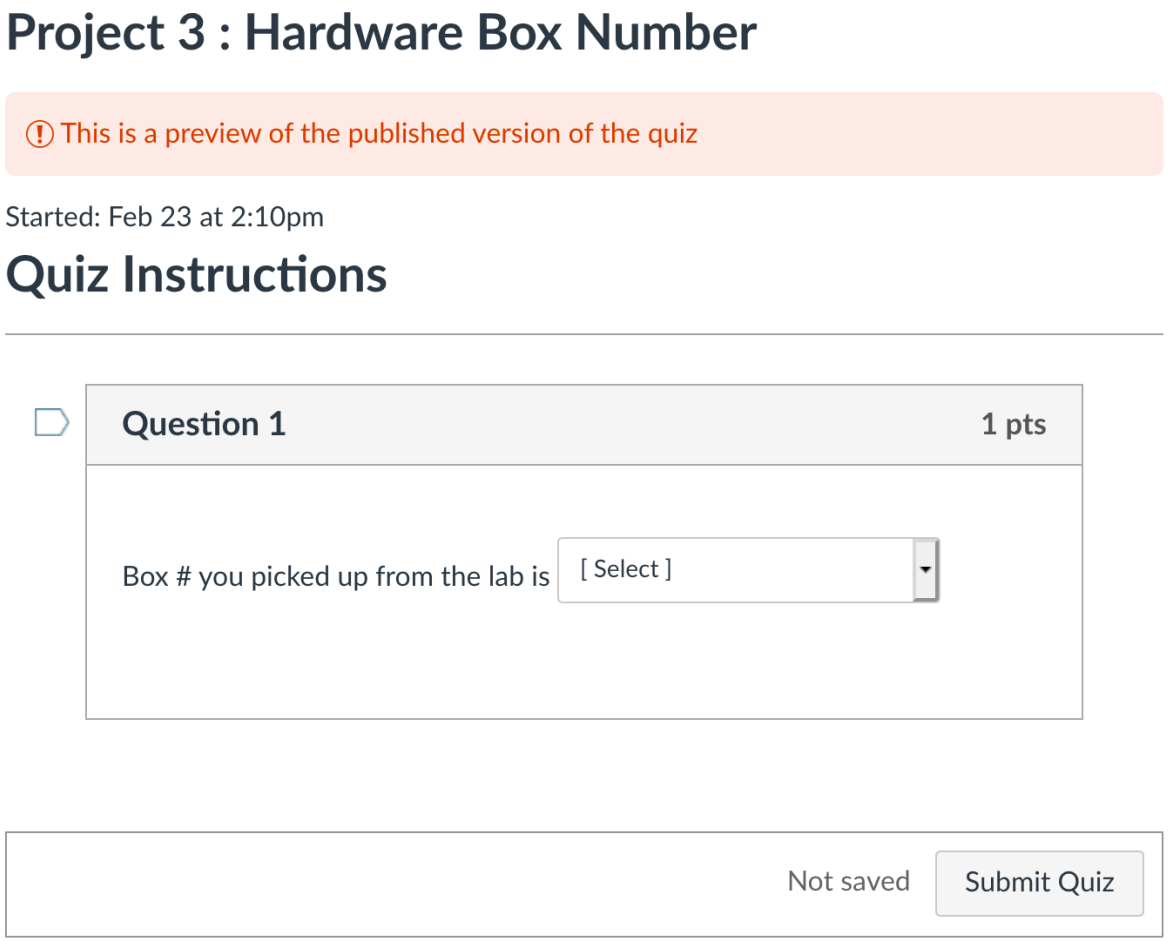
Andrew Lukefahr
Indiana University

Announcements

- P3 is out
- Hardware Boxes are ready
  - Password: ‘iuxilinx’

Register your Hardware

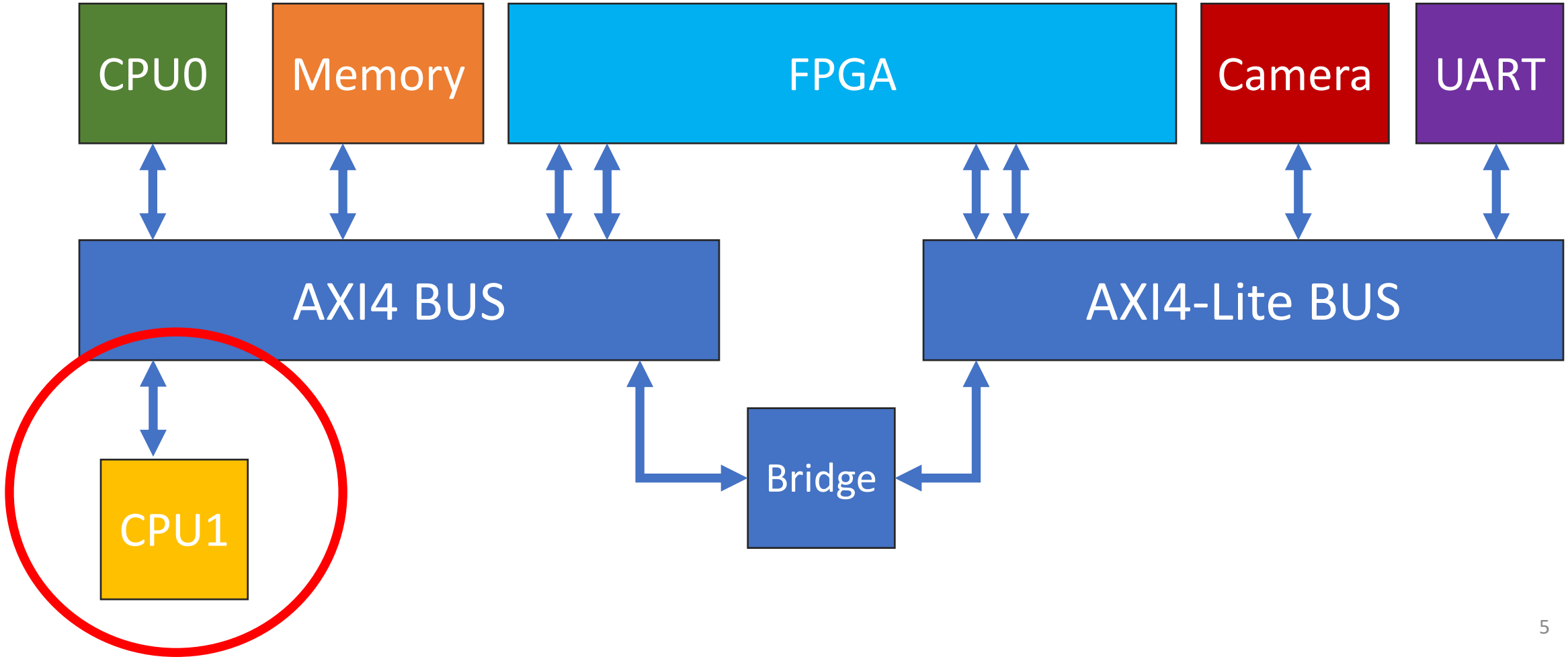
- Go to Canvas
- Select “P3 Box Number”
- Enter your box number



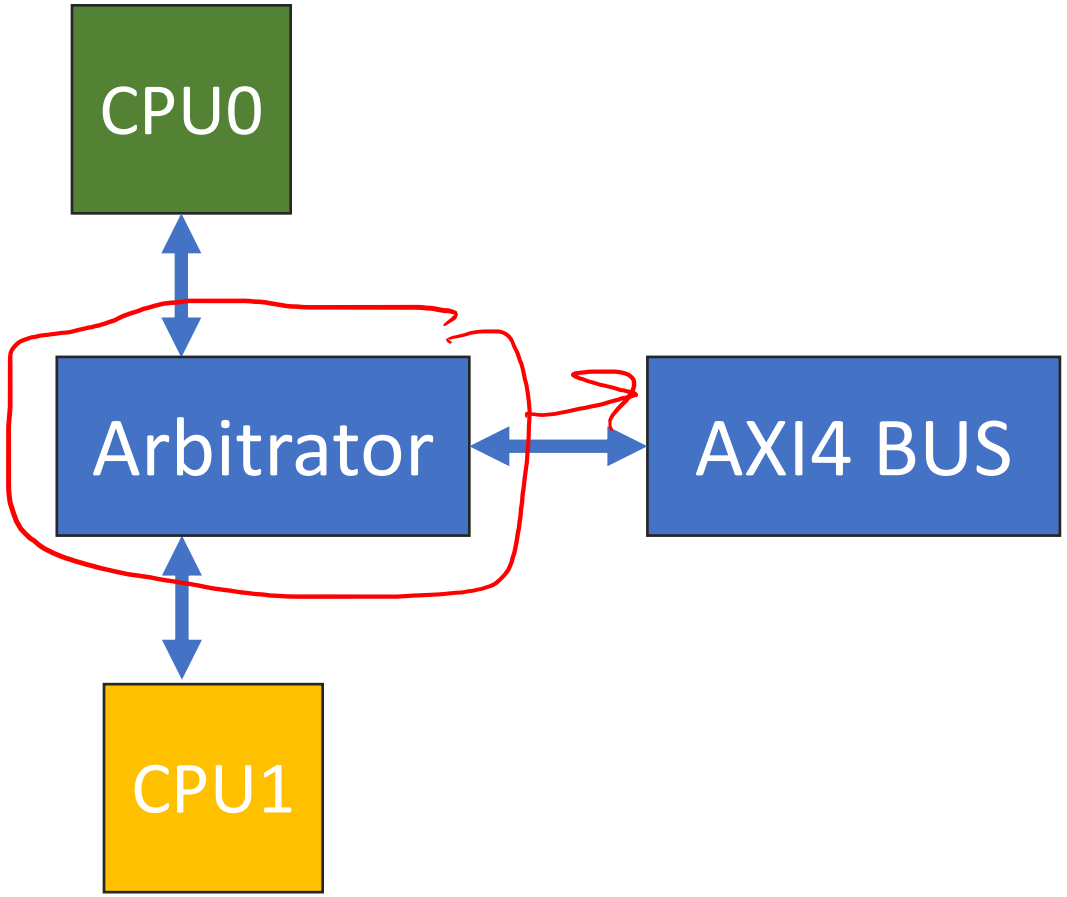
P3 Questions

- E315helper.py: Linux only.
- Simulations not finishing: Hit “Run All” button.
- Vivado: 2020.2 works.

Review: Multi-Master Buses



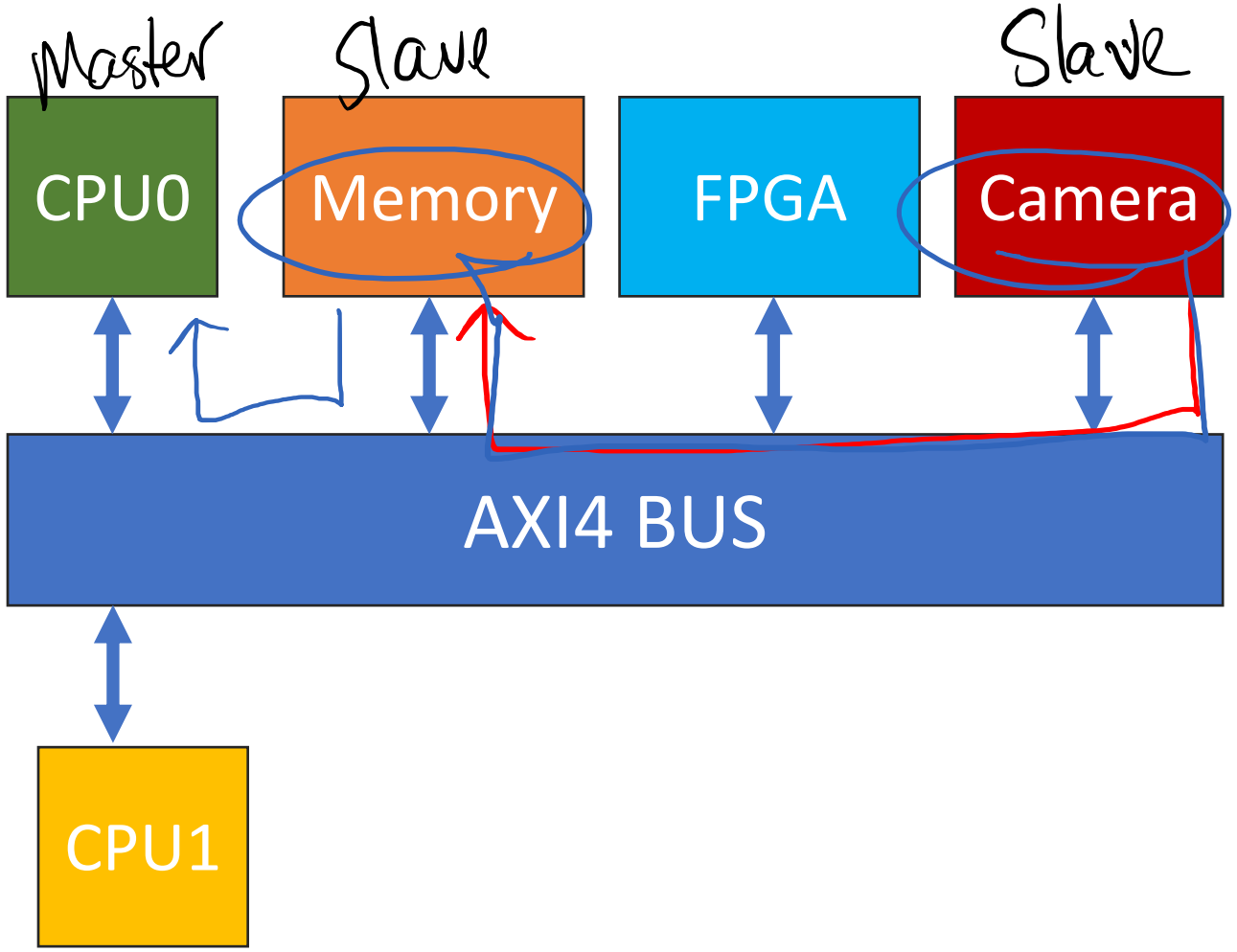
An Arbitrator selects who gets to use the bus



- What happens if both request a transaction at the same time?

- Arbitration:
  - Fixed-Priority
  - Round Robin
  - Many more…

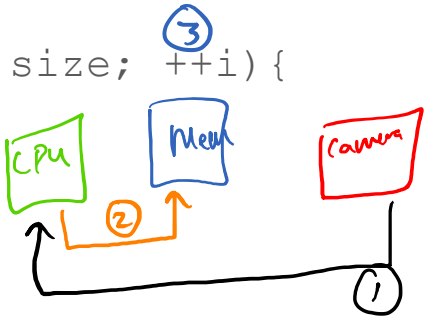
Q: How do I move data between the Camera and Memory?



A: The CPU copies data from Camera to Memory

```
#define CAMERA_MMIO_ADDR 0x40000004
volatile uint32_t * camera =
        (uint32_t *)(CAMERA_MMIO_ADDR);
#define BUF_SIZE 1024;
uint32_t buf[BUF_SIZE];

int main () {

    while (true){
        copy_image(camera, buf, BUF_SIZE);
        detect_face(buf);
    }
}
```

```
void copy_image (uint32_t * from,
                 uint32_t * to,
                 uint32_t size)
{
    register uint32_t reg;

    for (int i = 0; i < size; ++i){
    ①  reg = *from;

    ②  to[i] = reg;
    }④
}
```



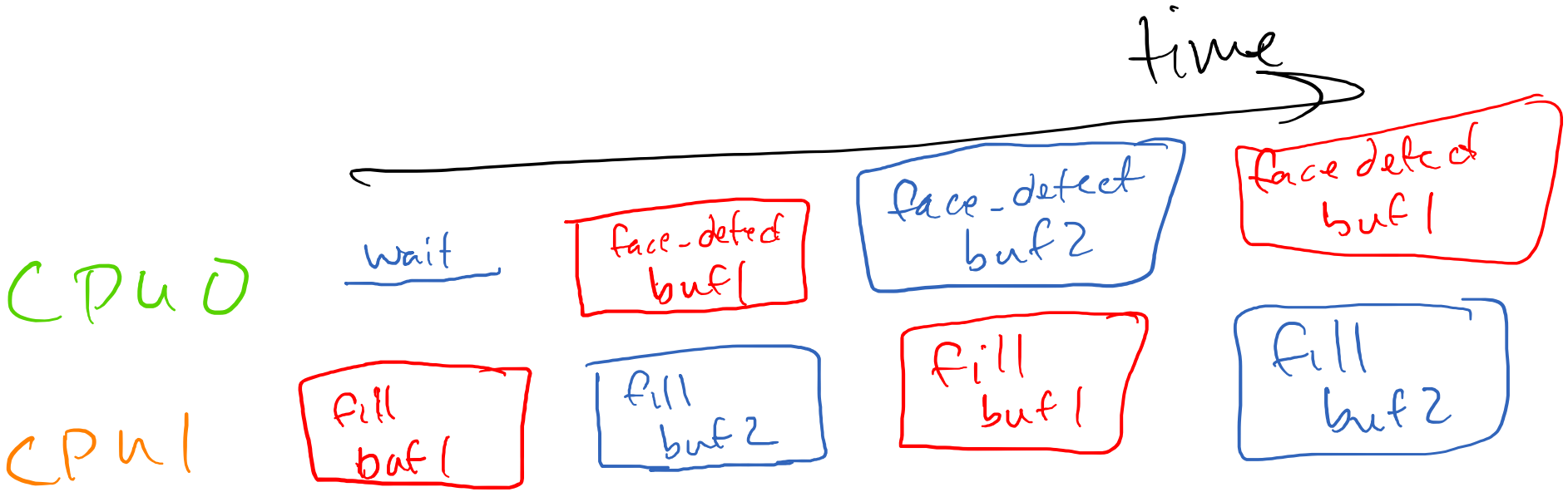
① load
② store
③ increment
④ branch

What if we do the copy on CPU1?

```
int main () {

    while (true){

        ask_cpu1_to_copy_image(camera, buf, BUF_SIZE);



        detect_face(buf);
    }
}
```

Copy on CPU1, Version 2.

```
int main () {

    ask_cpu1_to_copy_image(camera, buf1, BUF_SIZE);
    wait_for_cpu1_done();

    while (true){
        ask_cpu1_to_copy_image(camera, buf2, BUF_SIZE);
        detect_face(buf1);
        wait_for_cpu1_done();

        ask_cpu1_to_copy_image(camera, buf1, BUF_SIZE);
        detect_face(buf2);
        wait_for_cpu1_done();
    }

}
```

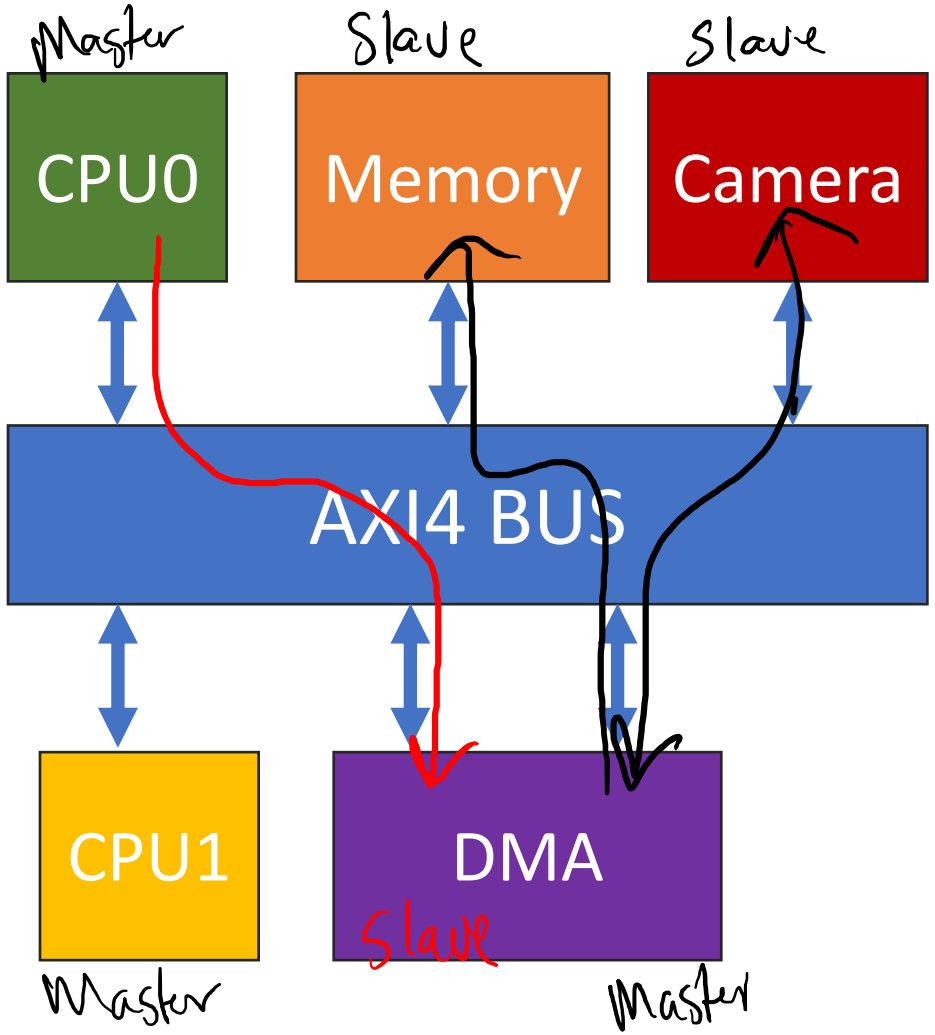

time
CPU 0
CPU 1
wait
face-detect buf1
face-detect buf2
face detect buf1
fill buf1
fill buf2
fill buf1
fill buf2

DMA: Direct Memory Access

- A mini-CPU that does copy for you:

```
void copy (uint32_t * from,
               uint32_t * to,
               uint32_t size)
{
   register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = *from;

        to[i] = reg;
    }
}
```

DMA has 2 interfaces



- Interface 1: Copy Memory
  - Data-Intensive Interface
  - AXI4 Master
  - Initiates Loads / Stores
- Interface 2: Tell DMA <u>what</u> to copy
  - Control Interface
  - AXI4 Slave
  - Responds to Loads/Stores

DMA

```
int main () {
    start
    dma_copy_image(camera, buf1, BUF_SIZE);
wait  wait_for_dma_done();

    while (true){
start   dma_copy_image(camera, buf2, BUF_SIZE);
        detect_face(buf1);
        wait_for_dma_done();

        dma_copy_image(camera, buf1, BUF_SIZE);
        detect_face(buf2);
        wait_for_dma_done();
    }
}
```

What do we need to do this?

start

```
void dma_copy (uint32_t * from,
               uint32_t * to,
               uint32_t size)
{
    register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = *from;

        to[i] = reg;
    }
}
```

Done?

What do we need to do this?

```
void dma_copy (uint32_t * from,
               uint32_t * to,
               uint32_t size)
{
   register uint32_t reg;

   for (int i = 0; i < size; ++i){

      reg = *from; //load

      to[i] = reg; //store
   }
}
```

- AXI4 Master Interface
  - Loads + Stores

- AXI4 Slave Interface
- 5 MMIO registers
  - Control (Start)
  - Status (Done)
  - Source (From)
  - Destination (To)
  - Size (in Bytes)

DMA V1.0 Interface

0x4000 - 4001
4002 4003

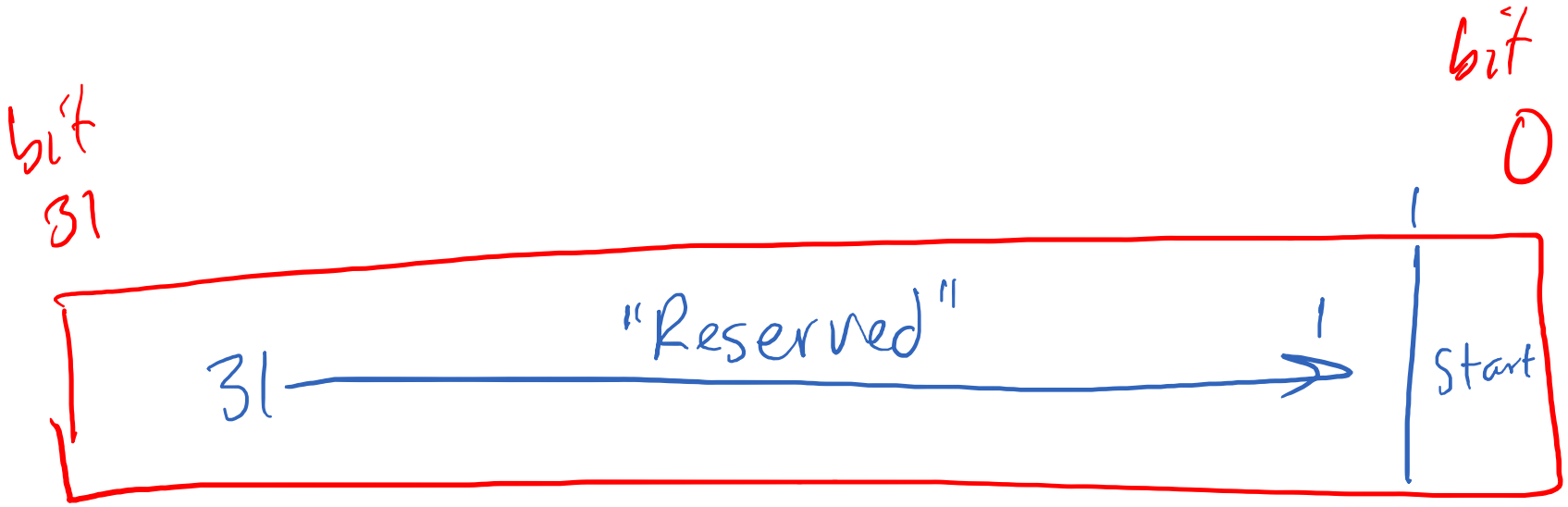
4 Bytes

- 0x0400: Control Register (Start)
- 0x0404: Status Register (Done)
- 0x0408: Source Address
- 0x040C: Destination Address
- 0x0410: Transfer Size in Bytes

(32-bit)
(4 Bytes)

Control Register

32-bit



Control Register

Control - 0x0400

31-1	0
Reserved	Start

Status Register

Control - 0x0400

31-1 Reserved	0 Start

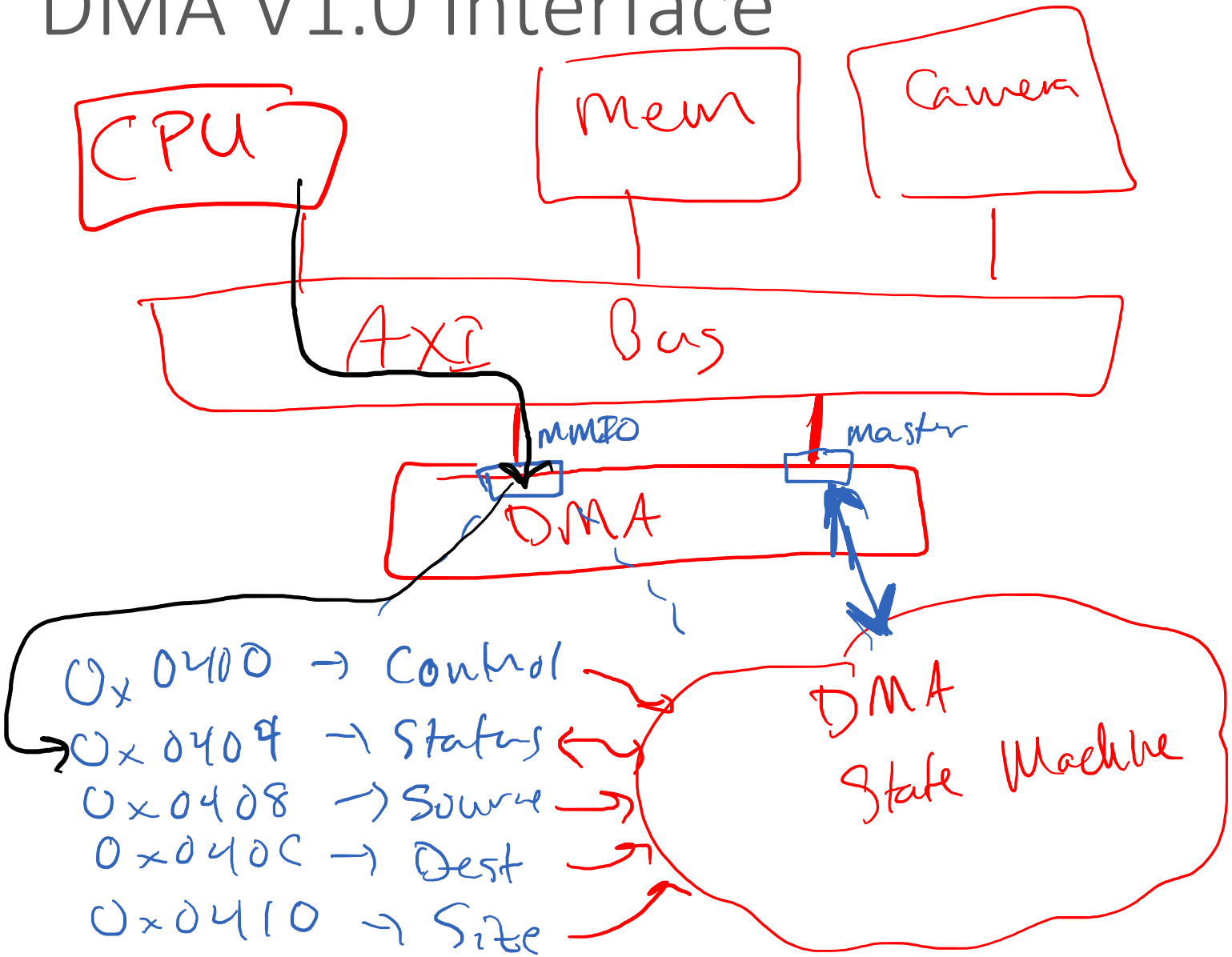
Status - 0x0404

31-1 Reserved	0 Done

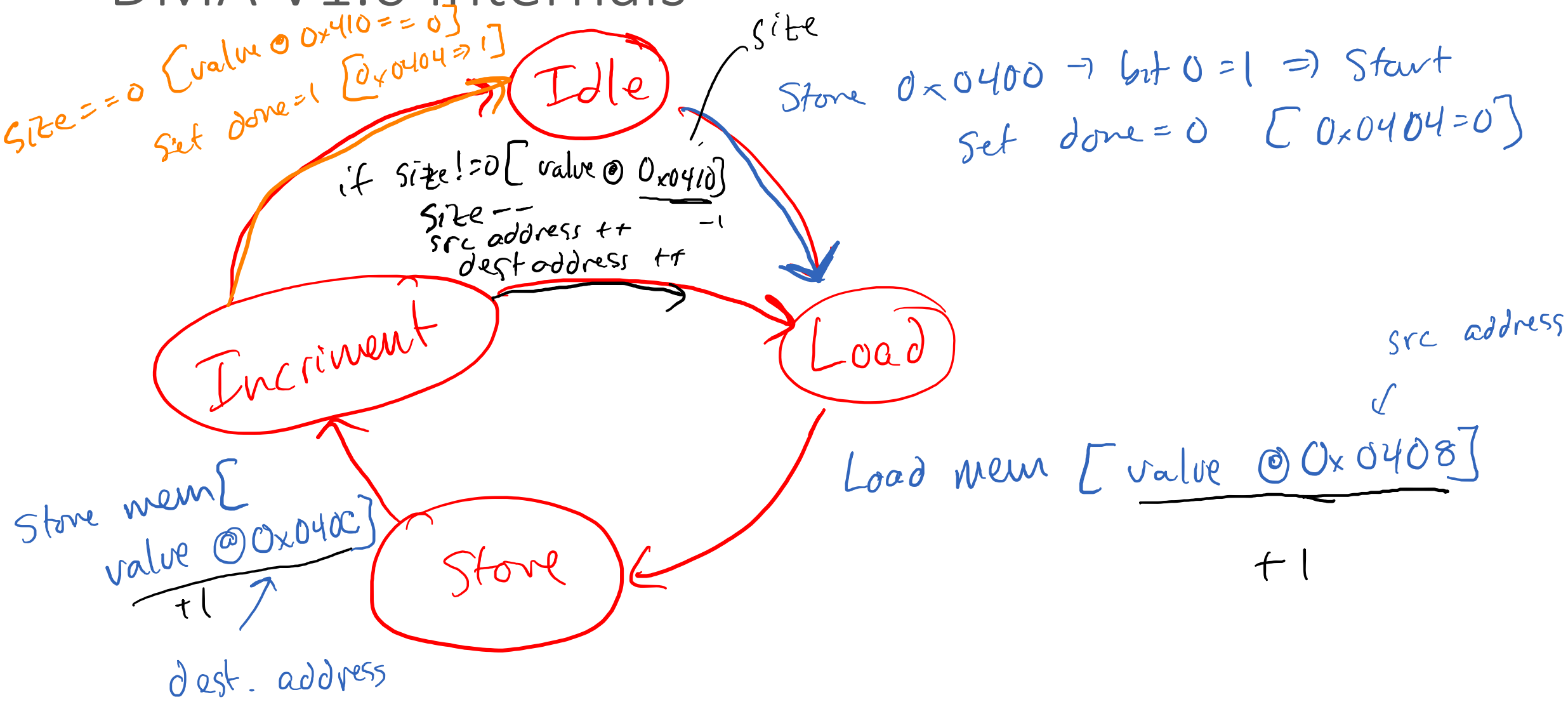
All Registers

Control - 0x0400	31-1 Reserved	0 Start
Status - 0x0404	31-1 Reserved	0 Done
Source - 0x0408	31-0 DMA Source Address "Camera"	
Destination - 0x040C	31-1 DMA Destination Address "buf1"	
Size - 0x0410	31-16 Reserved	15-0 DMA Transfer Size (in Bytes) 16-bits (2 Bytes)

DMA V1.0 Interface

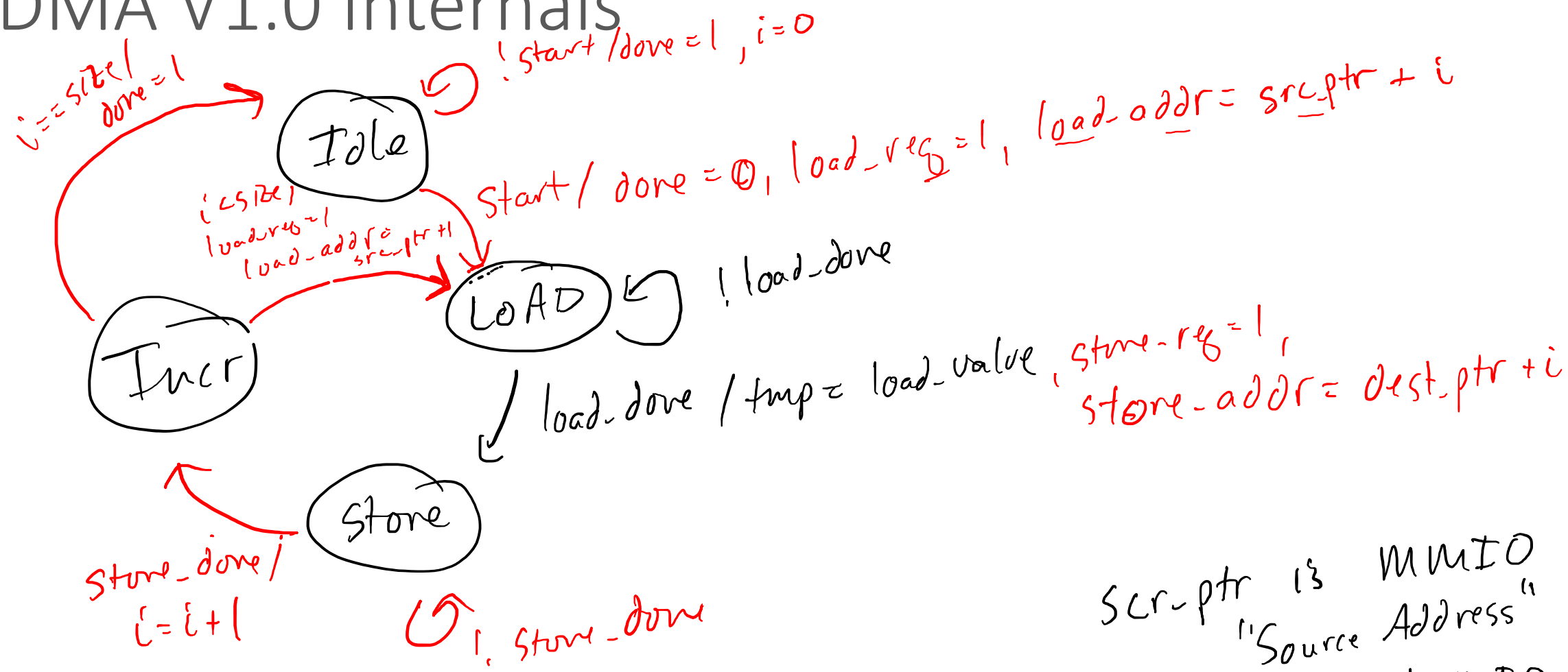


DMA V1.0 Internals



DMA V1.0 Internals

tmp = temporary value



scr_ptr is MMIO "Source Address"

dest_ptr is MMIO "Destination Address"

DMA V1.0 Internals

- IDLE: Status[Done]=1, wait for Control[Start]
- START: Status[Done] = 0, i = 0;
- LOAD: tmp = [Source+i]
- STORE: Dest+i = tmp
- BRANCH: i < TransferSize ? ++I, LOAD : IDLE

Does the AXI4 Full Interface have an address?

Does the AXI4 Full Interface have an MMIO Address?

- Is pretending to be memory, or a CPU?
- Does a CPU have a memory address?
- No.
- MMIO is for SLAVE interfaces.

Using DMA from the CPU:

0x0400: Control Register
0x0404: Status Register
0x0408: Source Address
0x040C: Destination Address
0x0410: Transfer Size in Bytes

```
void dma_copy ( uint32_t * src,
                uint32_t * dest,
                uint32_t size){



    //code me!

}
```

Using DMA from the CPU:

0x0400: Control Register
0x0404: Status Register
0x0408: Source Address
0x040C: Destination Address
0x0410: Transfer Size in Bytes

```
void dma_copy ( uint32_t * src,
                uint32_t * dest,
                uint32_t size){

    *((volatile uint32_t *)(0x0408))=src;
    *((volatile uint32_t *)(0x040C))=dest;
    *((volatile uint32_t *)(0x0410))=size;
    *((volatile uint32_t *)(0x0400))= 0x1; //start
    //spin until copy done?
    while( *((volatile uint32_t *)(0x0404)) != 0x1){;}
}
```

~~volatile keyword omitted~~!

Using DMA from the CPU:

0x0400: Control Register
0x0404: Status Register
0x0408: Source Address
0x040C: Destination Address
0x0410: Transfer Size in Bytes

Stopped here!

```
void dma_copy_start ( uint32_t * src,
                      uint32_t * dest,
                      uint32_t size){

    *((volatile uint32_t *)(0x0408))=src;
    *((volatile uint32_t *)(0x040C))=dest;
    *((volatile uint32_t *)(0x0410))=size;
    *((volatile uint32_t *)(0x0400))= 0x1; //start
}

void wait_for_dma(){
    //spin until copy done?
    while( *((uint32_t)(0x0404)) != 0x1){;}
}
```

*((volatile uint32t *) → 0x0408)

while(*(volatile uint32t *)(0x0404)) != 0x1) {;}

`volatile` keyword omitted!

Real DMA

Register Address Map

Table 2-6: **AXI CDMA Register Summary**

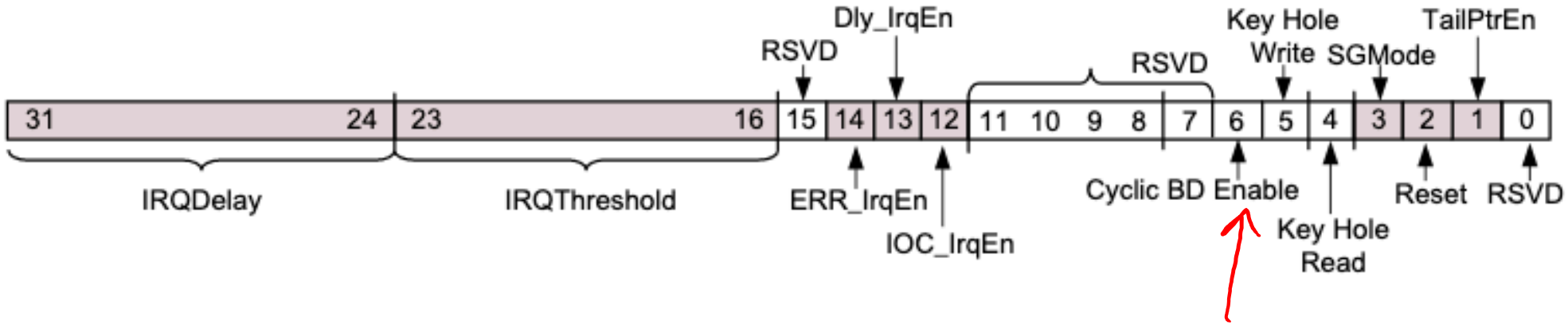
Address Space Offset[1]	Name	Description
00h	CDMACR	CDMA Control
04h	CDMASR	CDMA Status
08h	CURDESC_PNTR	Current Descriptor Pointer
0Ch[2]	CURDESC_PNTR_MSB	Current Descriptor Pointer. MSB 32 bits. Applicable only when the address space is greater than 32.
10h	TAILDESC_PNTR	Tail Descriptor Pointer
14h[2]	TAILDESC_PNTR_MSB	Tail Descriptor Pointer. MSB 32 bits. Applicable only when the address space is greater than 32.
18h	SA	Source Address
1Ch[2]	SA_MSB	Source Address. MSB 32 bits. Applicable only when the address space is greater than 32.
20h	DA	Destination Address
24h[2]	DA_MSB	Destination Address. MSB 32 bits. Applicable only when the address space is greater than 32.
28h	BTT	Bytes to Transfer

Real DMA

Register Details

CDMACR (CDMA Control – Offset 00h)

This register provides software application control of the AXI CDMA.



Other DMA tweaks

```
void dma_copy (uint32_t * from,
               uint32_t * to,
               uint32_t size,
               uint32_t inc_from, uint32_t inc_to)
{
   register uint32_t reg;

    for (int i = 0; i < size; ++i){

        reg = (inc_from ? *from[i] : *from);

        if (inc_to) to[i] = reg;
        else        to = reg;
    }
}
```

Other DMA tweaks

- Interrupts (not in E315)
- Repeat the transfer?

References

- https://www.xilinx.com/support/documentation/ip_documentation/axi_cdma/v4_1/pg034-axi-cdma.pdf

11: Direct Memory Access (DMA)

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University